Flutter + City Care

Guía práctica de conexión, autenticación y consumo de reportes

BACKEND	AUTENTICACIÓN	FLUJO CIUDADANO
Listo	**Token Bearer**	**Reportes propios**
0.0.0.0:3333	Almacenamiento seguro	MySQL + MinIO

Objetivo	Al terminar esta guía, la aplicación podrá iniciar sesión, conservar el token de forma segura, consultar categorías, enviar un reporte con fotografía y GPS, listar los reportes del ciudadano, abrir su detalle y cerrar sesión.

Backend verificado

- Cuenta ciudadana local creada con rol user y estado activo.
- Login, perfil, catálogo de 9 categorías y Mis reportes comprobados contra MySQL.
- Fotografías almacenadas en MinIO mediante URL firmada temporal.
- 65 pruebas funcionales aprobadas; contrato OpenAPI y cliente Orval actualizados.

Credencial local	El correo de desarrollo es ciudadano@citycare.com. La contraseña permanece únicamente en CITIZEN_INITIAL_PASSWORD dentro del archivo .env local y no se publica en este manual.

1. Direcciones y endpoints

Android Emulator no interpreta 127.0.0.1 como la computadora anfitriona. Dentro del emulador, 127.0.0.1 apunta al propio dispositivo virtual.

Método	Ruta	Uso en Flutter
POST	/api/v1/auth/login	Obtiene usuario y token.
GET	/api/v1/account/profile	Restaura la sesión.
POST	/api/v1/account/logout	Revoca el token actual.
GET	/api/v1/report-categories	Carga las categorías activas.
POST	/api/v1/reports	Envía reporte multipart con foto y GPS.
GET	/api/v1/reports	Lista únicamente reportes propios.
GET	/api/v1/reports/:id	Detalle, imagen, estado e historial.

URL según el entorno

Entorno	Base URL
Android Emulator	http://10.0.2.2:3333
Flutter web en la misma PC	http://127.0.0.1:3333
Dispositivo físico	http://IP_LOCAL_DE_LA_PC:3333
Producción	https://api.tu-dominio.com

Configuración del backend local

```
PORT=3333
HOST=0.0.0.0
APP_URL=http://127.0.0.1:3333
MINIO_PUBLIC_ENDPOINT=10.0.2.2
MINIO_PORT=9000
```

Importante Ejecuta el backend desde city-care con npm run dev. El contrato se consulta en http://127.0.0.1:3333/swagger y Swagger UI en /docs.

2. Preparar Flutter y Android

Instala las dependencias desde la raíz del proyecto Flutter. El comando elegirá versiones compatibles con tu SDK actual.

```
flutter pub add http
flutter pub add flutter_secure_storage
flutter pub add image_picker
```

Permiso de Internet

En android/app/src/main/AndroidManifest.xml, coloca el permiso antes de la etiqueta application.

```
<uses-permission android:name="android.permission.INTERNET" />

<application
    android:label="city_care"
    android:usesCleartextTraffic="true"
    android:icon="@mipmap/ic_launcher">
    ...
</application>
```

Solo desarrollo	usesCleartextTraffic permite HTTP local. En producción usa HTTPS y elimina esta excepción.

Estructura sugerida

```
lib/
  core/api/api_client.dart
  core/api/api_exception.dart
  core/storage/token_storage.dart
  features/auth/auth_service.dart
  features/reports/report_service.dart
  features/reports/models/report_category.dart
  features/reports/models/report_item.dart
```

Configuración central

```
class ApiConfig {
  static const String baseUrl =
      String.fromEnvironment(
        'CITY_CARE_API_URL',
        defaultValue: 'http://10.0.2.2:3333',
      );
}
```

Puedes cambiar el servidor sin editar código usando flutter run --dart-define=CITY_CARE_API_URL=http://IP_PC:3333.

3. Guardar el token de forma segura

No guardes el token en variables globales permanentes, SharedPreferences sin cifrar, archivos, logs ni capturas. flutter_secure_storage usa almacenamiento seguro del sistema operativo.

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';

class TokenStorage {
  static const _tokenKey = 'city_care_access_token';

  final FlutterSecureStorage _storage =
      const FlutterSecureStorage(
        aOptions: AndroidOptions(encryptedSharedPreferences: true),
      );

  Future<void> save(String token) {
    return _storage.write(key: _tokenKey, value: token);
  }

  Future<String?> read() {
    return _storage.read(key: _tokenKey);
  }

  Future<void> clear() {
    return _storage.delete(key: _tokenKey);
  }
}
```

Excepción uniforme

```
class ApiException implements Exception {
  final int statusCode;
  final String code;
  final String message;

  const ApiException(this.statusCode, this.code, this.message);

  @override
  String toString() => message;
}
```

Reglas de sesión

- Guardar el token solamente después de un login HTTP 200.
- Al abrir la app, leer el token y consultar /account/profile.
- Si profile devuelve 401 o 403, eliminar el token y volver al login.
- Al cerrar sesión, llamar primero al backend y limpiar el token incluso si falla la red.
- Nunca imprimir el token completo en debugPrint, consola o herramientas de analítica.

Duración	El backend controla la expiración mediante ACCESS_TOKEN_EXPIRES_IN. Flutter no debe inventar la fecha de vencimiento; debe reaccionar a HTTP 401/403.

4. Cliente HTTP y login

```
import 'dart:convert';
import 'package:http/http.dart' as http;

class ApiClient {
  ApiClient(this.tokens);
  final TokenStorage tokens;

  Uri uri(String path, [Map<String, dynamic>? query]) {
    return Uri.parse('${ApiConfig.baseUrl}$path')
        .replace(queryParameters: query?.map(
          (key, value) => MapEntry(key, '$value'),
        ));
  }

  Future<Map<String, String>> headers({bool auth = true}) async {
    final result = <String, String>{
      'Accept': 'application/json',
      'Content-Type': 'application/json',
    };
    if (auth) {
      final token = await tokens.read();
      if (token != null) result['Authorization'] = 'Bearer $token';
    }
    return result;
  }

  Map<String, dynamic> decode(http.Response response) {
    final body = response.body.isEmpty
        ? <String, dynamic>{}
        : jsonDecode(response.body) as Map<String, dynamic>;
    if (response.statusCode < 200 || response.statusCode >= 300) {
      final error = body['error'] as Map<String, dynamic>?;
      throw ApiException(
        response.statusCode,
        error?['code'] as String? ?? 'E_HTTP',
        error?['message'] as String? ?? 'Error de comunicación',
      );
    }
    return body;
  }
}
```

Servicio de autenticación

```
class AuthService {
  AuthService(this.api, this.tokens);
  final ApiClient api;
  final TokenStorage tokens;

  Future<Map<String, dynamic>> login(
    String email,
    String password,
  ) async {
    final response = await http.post(
      api.uri('/api/v1/auth/login'),
      headers: await api.headers(auth: false),
      body: jsonEncode({'email': email, 'password': password}),
    );
    final body = api.decode(response);
    final data = body['data'] as Map<String, dynamic>;
    await tokens.save(data['token'] as String);
    return data['user'] as Map<String, dynamic>;
  }
}
```

5. Restaurar sesión y consultar categorías

```
extension AuthRequests on AuthService {
  Future<Map<String, dynamic>?> restoreSession() async {
    if (await tokens.read() == null) return null;
    try {
      final response = await http.get(
        api.uri('/api/v1/account/profile'),
        headers: await api.headers(),
      );
      final body = api.decode(response);
      return body['data'] as Map<String, dynamic>;
    } on ApiException catch (error) {
      if (error.statusCode == 401 || error.statusCode == 403) {
        await tokens.clear();
        return null;
      }
      rethrow;
    }
  }
}
```

Modelo de categoría

```
class ReportCategory {
  final int id;
  final String code;
  final String name;
  final bool requiresDescription;

  const ReportCategory({
    required this.id,
    required this.code,
    required this.name,
    required this.requiresDescription,
  });

  factory ReportCategory.fromJson(Map<String, dynamic> json) {
    return ReportCategory(
      id: json['id'] as int,
      code: json['codigo'] as String,
      name: json['nombre'] as String,
      requiresDescription:
          json['requiereDescripcion'] as bool? ?? false,
    );
  }
}
```

Consulta del catálogo

```
Future<List<ReportCategory>> listCategories(ApiClient api) async {
  final response = await http.get(
    api.uri('/api/v1/report-categories'),
    headers: await api.headers(),
  );
  final body = api.decode(response);
  final data = body['data'] as List<dynamic>;
  return data
      .map((item) => ReportCategory.fromJson(
            item as Map<String, dynamic>))
      .toList();
}
```

6. Enviar un reporte con fotografía

El endpoint usa multipart/form-data. No establezcas Content-Type manualmente: MultipartRequest agrega el boundary correcto.

Valor que se envía	El endpoint de creación actual espera el nombre de la categoría en el campo categoria, por ejemplo Bache. Conserva también id y codigo para filtros y futuras versiones.

```
Future<Map<String, dynamic>> createReport({
  required ApiClient api,
  required String categoryName,
  required String photoPath,
  required double latitude,
  required double longitude,
  double? accuracy,
  String? address,
  String? description,
}) async {
  final token = await api.tokens.read();
  if (token == null) {
    throw const ApiException(401, 'E_NO_TOKEN', 'Inicia sesión');
  }

  final request = http.MultipartRequest(
    'POST',
    api.uri('/api/v1/reports'),
  );
  request.headers.addAll({
    'Accept': 'application/json',
    'Authorization': 'Bearer $token',
  });
  request.fields.addAll({
    'categoria': categoryName,
    'latitud': latitude.toString(),
    'longitud': longitude.toString(),
    'origenUbicacion': 'gps',
    if (accuracy != null)
      'precisionUbicacion': accuracy.toString(),
    if (address?.trim().isNotEmpty == true)
      'direccion': address!.trim(),
    if (description?.trim().isNotEmpty == true)
      'descripcion': description!.trim(),
  });
  request.files.add(await http.MultipartFile.fromPath(
    'foto',
    photoPath,
  ));

  final streamed = await request.send();
  final response = await http.Response.fromStream(streamed);
  return api.decode(response);
}
```

Validaciones del backend

- Fotografía obligatoria JPG, JPEG o PNG, máximo 10 MB.
- Latitud entre -90 y 90; longitud entre -180 y 180; 0,0 rechazado.
- origenUbicacion acepta gps o manual.
- La categoría Otro exige descripción.
- El folio se genera en el servidor; Flutter nunca debe inventarlo.
- El usuario del reporte se obtiene del token; no envíes userId.

7. Mis reportes, detalle y logout

Listado paginado

```
Future<Map<String, dynamic>> listMyReports(
  ApiClient api, {
  int page = 1,
  int perPage = 20,
  String? status,
  int? categoryId,
}) async {
  final response = await http.get(
    api.uri('/api/v1/reports', {
      'page': page,
      'perPage': perPage,
      if (status != null) 'status': status,
      if (categoryId != null) 'categoryId': categoryId,
    }),
    headers: await api.headers(),
  );
  return api.decode(response); // data + meta
}

Future<Map<String, dynamic>> getMyReport(
  ApiClient api,
  int id,
) async {
  final response = await http.get(
    api.uri('/api/v1/reports/$id'),
    headers: await api.headers(),
  );
  return api.decode(response);
}
```

Cerrar sesión

```
Future<void> logout(ApiClient api) async {
  try {
    final response = await http.post(
      api.uri('/api/v1/account/logout'),
      headers: await api.headers(),
    );
    api.decode(response);
  } finally {
    await api.tokens.clear();
  }
}
```

Campos principales de un reporte

Campo	Uso
folio	Identificador público CC-AÑO-000000.
estadoCodigo	pending, in_progress o resolved.
estado	Etiqueta visible en español.
fotoUrl	URL temporal para Image.network.
historial	Cambios de estado en el detalle.
creadoEn	Fecha ISO enviada por el backend.

8. Flujo final y comprobación

Imagen temporal	Cuando fotoUrl expire o Image.network falle, vuelve a consultar el listado o el detalle para obtener una URL firmada nueva. No conserves la URL como si fuera permanente.

Orden de inicialización

- Crear TokenStorage, ApiClient, AuthService y ReportService una sola vez mediante tu solución de estado.
- Mostrar pantalla de carga mientras restoreSession valida el token.
- Si existe sesión, cargar categorías y Mis reportes en paralelo.
- Después de crear un reporte, mostrar el folio y refrescar Mis reportes.
- Ante 401/403, limpiar sesión y regresar al login.

Prueba mínima de aceptación

Paso	Resultado esperado
1. Login	HTTP 200, token guardado y rol user.
2. Categorías	Se muestran 9 opciones activas.
3. GPS y foto	Coordenadas válidas y archivo JPG/PNG.
4. Envío	HTTP 201 y folio CC-AÑO-000000.
5. Persistencia	Texto en MySQL e imagen en MinIO.
6. Mis reportes	El folio nuevo aparece solo para ese ciudadano.
7. Detalle	Imagen, ubicación, estado e historial visibles.
8. Reinicio	La sesión se restaura mediante profile.
9. Logout	Token revocado y almacenamiento local vacío.

Errores frecuentes

- Connection refused: backend apagado, puerto incorrecto o HOST distinto de 0.0.0.0.
- 127.0.0.1 no responde: en Android Emulator usa 10.0.2.2.
- 401: token ausente, incorrecto, expirado o revocado.
- 403: cuenta inactiva o permiso no permitido.
- 422: revisa error.details, categoría, coordenadas, foto y descripción.
- 503: MinIO no está disponible; conserva el formulario para reintentar.

IA al final	MobileNetV2 no bloquea este flujo. Primero valida autenticación, reporte, MinIO y seguimiento. La IA se integrará después como sugerencia de categoría con nivel de confianza y revisión humana.